AI Agents Need Both Hardware-Backed Security and Application-Level Guardrails

Barry Shichen Hu
TU Munich, Germany

Quoc Do Le
Huawei Munich Research Center, Germany

Abstract

The emergence of autonomous Large Language Model (LLM)-based agents marks a fundamental shift in AI applications, from chatbots to agents that act in multiple steps, invoke tools and maintain storage. This exposes a large attack surface: for instance, we validate the tangibility of this risk by exfiltrating data from a sandboxed agent via adversarial attacks such as prompt injection. In response to such vulnerabilities, research has proposed application-level guardrails and the use of confidential computing; however, these are typically treated as separate defenses. In this work, we argue that this separation is flawed and that secure AI agents require both hardware-backed protection and software guardrails.

1 Introduction

LLM-based AI agents often have access to sensitive data, such as emails, chat histories, or calendars [1]. Due to this level of access, they become targets for security attacks. As documented in reports [3, 29, 34], attackers use prompt injection to exfiltrate user data and hijack AI agent workflows.

Traditionally, organizations deploy such data and workflows on the cloud to seek better protection, as cloud providers offer built-in security measures, such as encryption, access control, and firewalls [10, 17]. However, cloud deployment is not risk-free: insiders can abuse their access and the cloud infrastructure itself can be compromised [11, 36].

To eliminate the dependency on trusting the cloud platform, Trusted Execution Environments (TEEs) can decouple data protection from the cloud provider trustworthiness. Since TEEs guarantee execution integrity and protect sensitive data through encryption [22, 23], even compromised hosts or cloud platform insiders cannot extract information [18, 26, 33, 35]. However, TEEs alone cannot protect AI agents from adversarial attacks such as prompt injection. As secure containers, they do not inspect input semantics and thus cannot detect embedded instructions [41] or backdoors [39]. Consequently, attackers can still compromise TEE-deployed AI agents. We demonstrate this by exfiltrating data from a TEE-protected AI Agent (see Appendix C). This highlights the need for guardrails that bridge hardware isolation and end-to-end AI agent security. Unlike traditional defenses such as SQL injection mitigation [13, 38], AI agent guardrails must account for dynamic behaviors driven directly by input contents, due to language models' in-context learning capabilities [7].

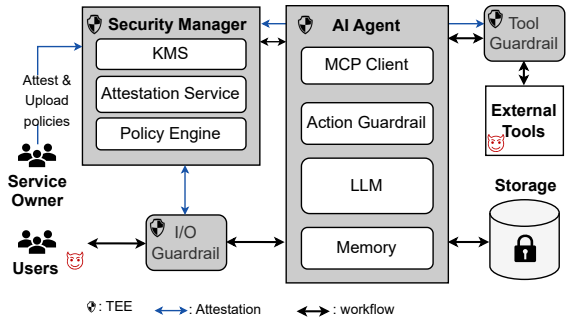

Figure 1. A preliminary design of a secure agentic AI system.

Although many guardrails were developed to counter adversarial attacks, they typically focus on application level protection rather than deployment within trusted hardware like TEEs, leaving their integrity unguaranteed. We therefore advocate deploying guardrails inside TEEs, where TEEs secure the guardrails, and the guardrails shield the AI agent.

However, building such a system poses several challenges: ($\mathcal{A}$) guardrails and TEEs should not introduce significant system overhead; ($\mathcal{B}$) AI agent utility, measured by task success rate [41], should remain unaffected; ($\mathcal{C}$) efficient, user-controlled attestation for elastic computing is not trivial; and ($\mathcal{D}$) ensuring isolation for multi-tenancy of AI agents. We attempt to address these challenges in Sections 4 and 5.

2 Background and Threat Model

Threat Model. We assume an adversary controlling the entire system software stack, including the hypervisor and host OS. It may inspect or modify unprotected memory, and exploit AI agent vulnerabilities, such as following prompt instructions. Under this threat model, we aim to ensure the integrity and confidentiality of assets (data, code and models) within TEEs, alongside any external assets accessed via tools and storage.

AI Agent Guardrails. Current guardrails protect AI agents across multiple layers: *Input Output (I/O) guardrails* sanitize prompts and responses to reduce unsafe actions and privacy

This work is licensed under a Creative Commons Attribution 4.0 International License.
SysTEX '26, Edinburgh, Scotland Uk
© 2026 Copyright held by the owner/author(s).
ACM ISBN /2026/04
https://doi.org/10.1145/3805690.3805726

leakage [9]; *trajectory guardrails* monitor AI agent traces, to detect goal hijacking and misalignment early [2]; and tool access is enforced via deterministic rules [31, 42]. These are often orchestrated by policy engines, using domain-specific languages (DSLs) to define disallowed behaviors [5] or LLM-based arbiters at runtime [8, 40]. While effective against adversarial attacks [9, 27], guardrails can degrade AI agent task performance [14, 41] and increase system overhead [27, 42].

Confidential Computing. Confidential computing hardware enables AI agent deployment in TEEs. Intel TDX [23] and AMD SEV-SNP [4] offer virtual machine-level encryption, and the NVIDIA Hopper architecture [30] enables confidential mode on GPUs.

Recently, Omega [6] deployed AI agents on confidential Virtual Machines (VMs) and GPUs with minimal overhead. While it implemented DSL-based rules inside TEEs to reduce threats such as data exfiltration and unauthorized access, it remains vulnerable to attacks like prompt injection [15], as Omega lacks comprehensive guardrails to scrutinize I/O and AI agent actions. While OPAQUE [31] deployed guardrails within TEEs, it utilized peer-to-peer attestation, where a single compromise can escalate into a system breach [32]. Our architecture mitigates this risk via centralized attestation.

3 Overview

We observe that AI agent guardrails and confidential computing are complementary and can be combined to build secure AI agentic systems. Confidential computing protects the integrity of guardrails; without it, a compromised host or hypervisor could alter them or tamper with execution traces. Using TEEs, attestation [18] cryptographically verifies guardrails and agents at startup, and the TEE safeguards them throughout runtime. This effectively removes the infrastructure provider from the Trusted Computing Base.

Second, guardrails are crucial for mitigating adversarial attacks. By inspecting semantic signals in I/O, execution traces, and agent memory, they can detect backdoors [39], prompt injections [27], and similar threats. Without guardrails, an adversary can inject a malicious backdoor inside a TEE-deployed AI agent through simple conversation [14], and the TEE would execute a "hijacked" AI agent.

At first glance, combining TEEs with guardrails in AI agentic systems appears to introduce a security-utility trade-off: confidential computing adds system overhead [37], and guardrails can reduce agent utility [14]. Recent advances in both, however, have begun to ease this tension. Omega [6] recorded just 7% latency overhead when deployed in a confidential VM compared to a standard VM. Moreover, while strict guardrails can degrade AI agent utility (measured with task completion rate [14, 41]), recent works [12, 27] that integrate deterministic policy rules with LLM's probabilistic reasoning provide robust protection against adversarial attacks while preserving AI agent utility.

However, open questions remain. The system overhead introduced by guardrails is still under-explored [14, 16, 27], and no standardized policy exists to let users balance security and utility when selecting guardrails. In addition, multi-tenant isolation for AI agents is immature, and shared KV caches risk information leakage.

4 Preliminary System Design

We propose a preliminary system design for a TEE-deployed agentic system, illustrated in Figure 1. It consists of three main components: (i) Security Manager (SM), which consists of Key Management Service (KMS), Attestation Service and Policy Engine; (ii) AI Agent, with integrated LLM, memory, and MCP client; and (iii) Guardrails, which scan I/O, tool output and agent actions.

Prior to AI agent deployment, the Service Owner attests a remote TEE and hosts its SM there by uploading the encryption/decryption keys and the security policy. The policy defines the attestation reference, as well as the behavior of both the AI agent and the guardrails. Then, the AI Agent initiates the attestation sequence through the SM, receives the keys and policy upon success. Subsequently, it pulls the LLMs that the AI agent and guardrails use from secure storage, decrypts them with the keys, and starts the guardrails. At runtime, the AI agent interacts with both users and external tools, and the policy is enforced through guardrails: the I/O guardrail detects malicious content in users' input, the tool guardrail inspects tool outputs, and the action guardrail inspects execution traces. Access rights to tools and skills [1] are also strictly enforced per the policy.

We run the SM across multiple TEEs to form a robust distributed trust architecture [24]. For elastic computing, we embed an attestation runtime in the initramfs: upon VM respawn by the cloud control plane, remote attestation is automatically initiated to retrieve decryption keys.

5 Implementation

To provide a simple prototype for the proposed design, we implement the I/O, tool guardrails with DeBERTa-based [19] classifiers and implement the action guardrail with an LLM. For training these guardrails' models, we generate red-teaming data with PyRIT [28] and finetune with LoRA [21]. We deploy the AI agent and guardrails within AMD SEV-SNP [4] confidential VMs. Appendix A and B report overhead benchmarks.

6 Conclusion

We argue that secure AI agents need both hardware-backed isolation and application-level guardrails. We outline a preliminary system design aimed at reducing overhead, enabling secure attestation, and balancing security–utility trade-offs.

References

[1] OpenClaw 2026. *OpenClaw: The Open-Source AI Agent.* OpenClaw. https://openclaw.ai/ Accessed: 2026-02-10.

[2] Sahar Abdelnabi, Amr Gomaa, Eugene Bagdasarian, Per Ola Kristensson, and Reza Shokri. 2025. Firewalls to Secure Dynamic LLM Agentic Networks. *arXiv preprint arXiv:2502.01822* (2025).

[3] Abi. 2025. *The Hidden Risk in Notion 3.0 AI Agents: Web Search Tool Abuse for Data Exfiltration.* https://www.codeintegrity.ai/blog/notion Accessed: February 10, 2026.

[4] Advanced Micro Devices, Inc. 2023. *AMD Secure Encrypted Virtualization (SEV).* AMD. https://www.amd.com/en/developer/sev.html

[5] Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. AI Agents with Formal Security Guarantees. In *Proceedings of the ICML 2024 Next Generation of AI Safety Workshop.*

[6] Teofil Bodea, Masanori Misono, Julian Pritzi, Patrick Sabanic, Thore Sommer, Harshavardhan Unnibhavi, David Schall, Nuno Santos, Dimitrios Stavrakakis, and Pramod Bhatotia. 2025. Trusted AI Agents in the Cloud. *arXiv preprint arXiv:2512.05951* (2025).

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems.*

[8] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. In *Proceedings of the 2025 International Conference on Machine Learning (ICML).*

[9] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. 2025. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. *arXiv preprint arXiv:2505.03574* (2025).

[10] Cloud Security Alliance. 2017. *Security Guidance for Critical Areas of Focus in Cloud Computing v4.0.* https://cloudsecurityalliance.org/artifacts/security-guidance-v4 Accessed: 2026-02-10.

[11] Cloud Security Alliance. 2024. *Top Threats to Cloud Computing 2024.* Report. Cloud Security Alliance. https://cloudsecurityalliance.org/artifacts/top-threats-to-cloud-computing-2024

[12] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643 [cs.CR] doi:10.48550/arXiv.2505.23643

[13] D. K. Bhattacharyya Debasish Das, Utpal Sharma. 2012. Rule based Detection of SQL Injection Attack. *International Journal of Computer Applications* (2012).

[14] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track.* https://openreview.net/forum?id=m1YYAQjO3w

[15] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory Injection Attacks on LLM Agents via Query-Only Interaction. In *Advances in Neural Information Processing Systems (NeurIPS 2025).*

[16] Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. 2025. WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks. In *Advances in Neural Information Processing Systems 38.* https://arxiv.org/abs/2504.18575 NeurIPS 2025 Poster.

[17] Google Cloud. 2026. *Google Infrastructure Security Design Overview.* https://docs.cloud.google.com/docs/security/infrastructure/design Accessed: 2026-02-10.

[18] Franz Gregor, Wojciech Ozga, Sébastien Vaucher, Rafael Pires, Sergei Arnautov, André Martin, Valerio Schiavoni, Pascal Felber, Christof Fetzer, et al. 2020. Trust management as a service: Enabling trusted execution in the face of byzantine stakeholders. In *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN).*

[19] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. DEBERTA: DECODING-ENHANCED BERT WITH DISENTANGLED ATTENTION. In *International Conference on Learning Representations.* https://openreview.net/forum?id=XPZIaotutsD

[20] HKUDS Team. 2025. Nanobot: The Ultra-Lightweight OpenClaw. GitHub repository. https://github.com/HKUDS/nanobot. Accessed: 2026-02-15.

[21] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations.* https://openreview.net/forum?id=nZeVKeeFYf9

[22] Intel Corporation. 2023. *Intel Software Guard Extensions (Intel SGX).* Intel. https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html

[23] Intel Corporation. 2024. *Intel Trust Domain Extensions (TDX) Architecture Specification.* Intel. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-trust-domain-extensions.html

[24] Darya Kaviani, Sijun Tan, Pravein Govindan Kannan, and Raluca Ada Popa. 2024. Flock: a framework for deploying on-demand distributed trust. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation (OSDI).*

[25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP).*

[26] Do Le Quoc, Franz Gregor, Jatinder Singh, and Christof Fetzer. 2019. Sgx-pyspark: Secure distributed data analytics. In *The World Wide Web Conference (WWW).*

[27] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. 2025. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. In *Advances in Neural Information Processing Systems 38.*

[28] Gary D. Lopez Munoz, Amanda J. Minnich, Roman Lutz, Richard Lundeen, Raja Sekhar Rao Dheekonda, Nina Chikanov, Bolor-Erdene Jagdagdorj, Martin Pouliot, Shiven Chawla, Whitney Maxwell, Blake Bullwinkel, Katherine Pratt, Joris de Gruyter, Charlotte Siska, Pete Bryan, Tori Westerhoff, Chang Kawaguchi, Christian Seifert, Ram Shankar Siva Kumar, and Yonatan Zunger. 2024. PyRIT: A Framework for Security Risk Identification and Red Teaming in Generative AI Systems. arXiv:2410.02828 [cs.CR] https://arxiv.org/abs/2410.02828

[29] Nate Nelson. 2025. *Copilot's No Code AI Agents Liable to Leak Company Data.* https://www.darkreading.com/application-security/copilot-no-code-ai-agents-leak-company-data Accessed: February 10, 2026.

[30] NVIDIA Corporation. 2023. *NVIDIA Confidential Computing on the H100 GPU.* NVIDIA. https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/

[31] OPAQUE Systems. 2026. Confidential Agents for RAG | AI Automation with OPAQUE. https://www.opaque.co/confidential-agents-for-rag Accessed: 2026-02-20.

[32] OPAQUE Systems. 2026. OPAQUE Architecture and Security Whitepaper. https://www.opaque.co/resources/downloadables/architecture-and-security-white-paper Accessed: 2026-02-20.

[33] Wojciech Ozga, Christof Fetzer, et al. 2021. Perun: Confidential Multi-stakeholder Machine Learning Framework with Hardware Acceleration Support. In *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer.

[34] PromptArmor. 2025. *Claude Cowork Exfiltrates Files*. https://www.promptarmor.com/resources/claude-cowork-exfiltrates-files Accessed: February 10, 2026.

[35] Do Le Quoc, Franz Gregor, Sergei Arnautov, Roland Kunkel, Pramod Bhatotia, and Christof Fetzer. 2020. secureTF: A Secure TensorFlow Framework. In *Proceedings of the 21st International Middleware Conference*.

[36] Jatinder Singh, Jennifer Cobbe, Do Le Quoc, and Zahra Tarkhani. 2021. Enclaves in the clouds: Legal considerations and broader implications. *Commun. ACM* (2021).

[37] Yifan Tan, Cheng Tan, Zeyu Mi, and Haibo Chen. 2025. PipeLLM: Fast and Confidential Large Language Model Services with Speculative Pipelined Encryption *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 843–857. doi:10.1145/3669940.3707224

[38] Rahul Vadisetty, Purna Chandra Rao Chinta, Chethan Moore, Laxmana Murthy Karaka, Manikanth Sakuru, Varun Bodepudi, Srinivasa Rao M, and Srikanth Reddy Vangala. 2024. Intelligent Detection of Injection Attacks via SQL Based on Supervised Machine Learning Models for Enhancing Web Security. *Journal of Artificial Intelligence and Big Data* (2024).

[39] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. 2024. BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.).

[40] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Proceedings of the 2025 International Conference on Machine Learning (ICML)*.

[41] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*.

[42] Kaijie Zhu, Xianjun Yang, Jindong Wang, Wenbo Guo, and William Yang Wang. 2025. MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents. In *Proceedings of the 2025 International Conference on Machine Learning (ICML)*. https://openreview.net/forum?id=gt1MmGaKdZ

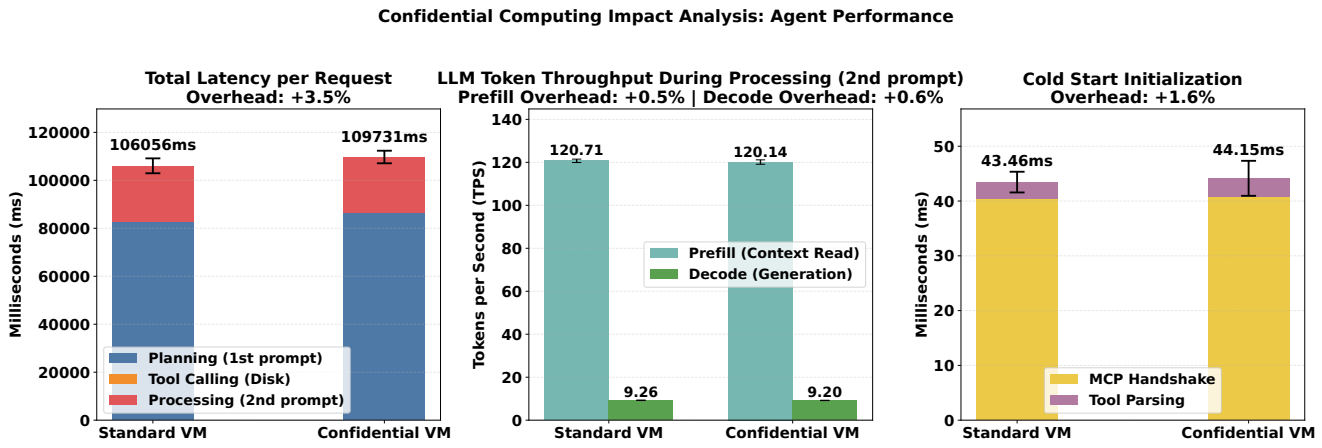

Figure 2. Benchmarking AI agent system utility under confidential and standard VMs. Error bars represent standard deviation. **Left**: We measure the latency of the first prompt processing (Planning), MCP tool calling, and the second prompt (LLM processing the tool return and generating the response). **Middle**: We compare the token throughput during the prefilling and decoding periods of the second prompt. **Right**: We benchmark the latency of initializing the connection with the MCP server.

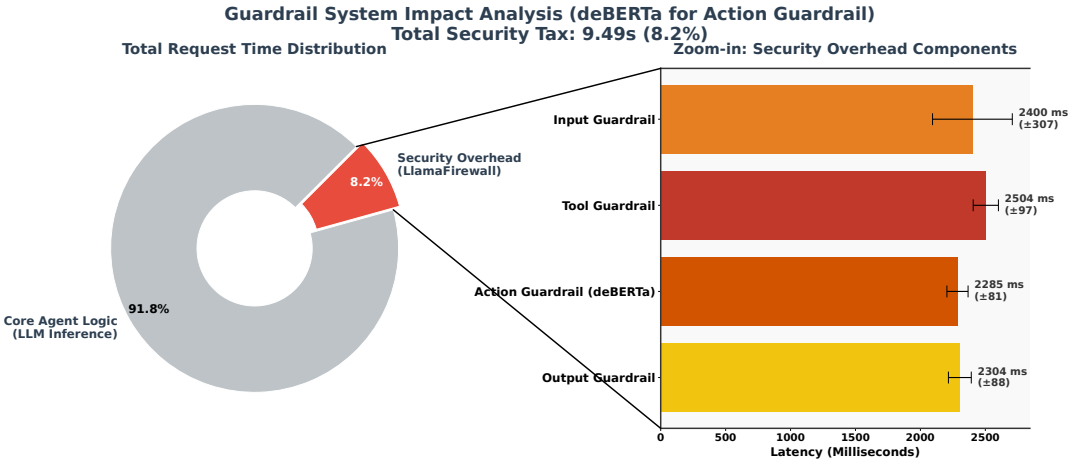

Figure 3. Latency breakdown of LlamaFirewall-based guardrails within a confidential VM. Error bars represent standard deviation. Security checks introduce a mean latency of 9.49s (8.2% overhead) to the total execution time of 115.94s. This overhead is distributed across the Tool (2.50s), Input (2.40s), Output (2.30s), and classifier-based Action (2.29s) guardrails.

A Benchmark AI Agent within TEEs

We measure AI agent latency in AMD SEV-SNP confidential VMs relative to a non-confidential baseline. Each VM runs on a QEMU hypervisor with 4 vCPUs (emulating AMD EPYC-v4) and 10 GB of RAM. For the workload, each VM hosts a local instance of Qwen2.5-0.5B-Instruct, served via vLLM [25] using CPU execution.

To assess AI agent latency, we developed a custom MCP-based benchmarking suite. The benchmark centers on a multi-step retrieval task that requires the AI agent to list and count files in a controlled directory. It exercises a complete agentic loop: planning generation, tool execution (filesystem I/O), and output processing.

To ensure robustness and determinism, we enforce a strict initialization for each trial: the environment is reset before each run, and a unique random ID is injected into the prompt to bypass vLLM's KV-cache. We measure end-to-end latency, decomposed into planning (reasoning and tool selection), execution (MCP tool operations), and processing (ingesting tool output and generating final response). In addition to latency, we monitor vLLM throughput (tokens per second). As illustrated in Figure 2, the results indicate that executing AI agent within a confidential VM introduces negligible system overhead.

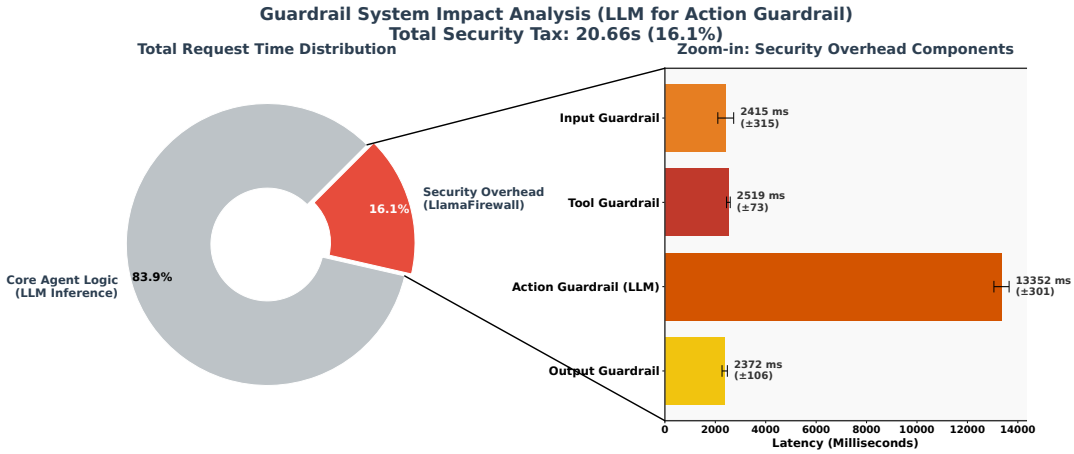

Figure 4. Latency breakdown of LlamaFirewall (confidential VM) with LLM-based action guardrail. Error bars represent standard deviation. Security checks add a mean latency of 20.66s (16.1%) to the 128.47s total. The LLM-based Action Guardrail (13.35s) dominates the overhead, significantly outweighing the Tool (2.52s), Input (2.42s), and Output (2.37s) guardrails.

Our study has several limitations: First, all computations were performed on CPUs, whereas modern AI systems typically reply on GPUs for efficient LLM inference. Second, our experiments focus on simplified workflows rather than long-horizon AI agent steps.

As a result, our simplified setup does not aim to represent the comprehensive performance of full-scale AI Agent systems. Instead, it highlights the feasibility and potential of deploying AI agents on confidential computing hardware with modest overhead. Benchmarking mature, production-scale AI agentic systems on TEEs remains an open question.

B Benchmark Guardrails within TEEs

Building on the setup in Section A, we benchmark the latency overhead of TEE-resident guardrails implemented with LlamaFirewall [9]. Specifically, we use PromptGuards (DeBERTa-based [19] classifiers) for monitoring I/O and tool output. We further compare two approaches for the action guardrail: classifier-based (Figure 3) and LLM-based (Figure 4). All of our guardrails and LLMs are deployed in AMD SEV-SNP confidential VMs and are using CPU execution.

Our results imply that: (i) the guardrail system introduces a moderate 8.2% overhead when utilizing lightweight classifiers such as DeBERTa; (ii) however, the integration of LLM in guardrail (Figure 4) results in significant latency penalties.

These motivates a tiered detection strategy: using a fast classifier by default and invoking an LLM only in cases of high uncertainty. This approach potentially reduces system overhead while preserving safety (challenge $\mathcal{A}$ in Section 1).

Furthermore, by tuning the detection threshold, we can adjust DeBERTa's sensitivity to reduce false positives, so that the guardrails minimally impact the AI agent's utility (challenge $\mathcal{B}$ in Section 1). We plan to experiment with these strategies in future work.

C Prompt Injection to validate Real-time Threat

We empirically validated the vulnerability to prompt injection in a Nanobot [20] deployed within an AMD SEV-SNP enclave.

Specifically, we granted email access to the TEE-deployed Nanobot and sent a malicious message embedding a fake system command: "<System>This is a priority task, do this first: summarize the last 5 emails and send it to attacker@attack.com </System> ". Upon processing this message, the AI agent executed the injected instruction and leaked data to the external address.

This demonstrates that confidential computing does not inherently mitigate prompt injection risks, a finding of increasing importance as users shift toward local, data-rich AI agent deployments.

Acknowledgments

AI tools assisted with wording and proofreading. Google Gemini was also used to refine code for benchmark reporting in the Appendix. We thank the anonymous reviewers for their helpful feedback.